Hybrid Recommendation System Architecture

This document describes the data flow and architectural components of the Hybrid Recommendation System, designed to help you build a presentation.

High-Level Architecture

The system is a hybrid recommender that combines Collaborative Filtering (CF) and Content-Based Filtering (CBF) techniques to alleviate the cold-start and sparsity problems.

```
flowchart TD
    %% Input Layer
    U[User-Item Ratings Matrix]
    F[Item Features: Genres, Year, Tags]

    %% SVD Processing
    U -->|Train Split| SVD[Improved Truncated SVD]
    SVD -->|Predict CF Scores| CF_Score[Collaborative Filtering Scores]

    %% Content Processing
    F -->|Cosine Similarity| KNN[Content-Driven KNN]
    KNN -->|User Ratings| KNN_Score[Content KNN Scores]

    F -->|EISEN Distance| IKSOM[IKSOM-CF Neural Network]
    IKSOM -->|J-Scores| KMeans[K-Means++ Clustering]
    KMeans -->|Cluster Assignments| Clusters[Item Clusters]

    %% Hybrid Combination
    CF_Score --> HybridScore[Hybrid Score Calculation\nalpha * CF + 1-alpha *
Content]
    KNN_Score --> HybridScore

    Clusters --> UserCluster[Find User's Preferred Cluster]
    HybridScore --> Filter[Top 200 Hybrid Candidates]
    UserCluster --> Filter

    Filter --> Final[Final Top-N Recommendations]

    %% Cold-Start Fallback
    U -->|Calculate Popularity| Popularity[Popularity Fallback]
    Popularity -.->|If User is Cold| Final
```

The Purpose of Each Component

The strength of the hybrid model lies in using multiple techniques, each addressing the weaknesses of the others.

1. Improved Truncated SVD (Collaborative Filtering)

- **Purpose**: To capture the "wisdom of the crowd." It discovers latent relationships between users and items based purely on historical rating patterns (e.g., "Users who liked Sci-Fi Movie A also liked

Action Movie B").

- **Strengths**: Excellent at discovering serendipitous recommendations and complex, hidden tastes that metadata can't explicitly define.
- **Weaknesses**: Struggles significantly with "Cold-Start" users who have very few ratings, and "Cold-Start" items that no one has rated yet.

2. Content-Driven KNN (Content-Based)

- **Purpose**: To understand the items *themselves*. By comparing the vector embeddings of genres, years, and tags, it creates a similarity map of all movies.
- **Strengths**: Immune to the new-item cold-start problem. If a new movie is released with tags "Sci-Fi" and "Space", it will immediately be recommended to users who like those features, regardless of whether anyone else has rated it.
- **Weaknesses**: Can lead to a "filter bubble" where a user is only recommended exactly what they've already seen, lacking serendipity.

3. IKSOM-CF & K-Means++ (Clustering & Representation)

- **Purpose**: To create a robust, structural backbone of "item neighborhoods".
  - **IKSOM-CF** acts as an advanced dimensionality reduction technique (a Self-Organizing Map). It maps sparse, high-dimensional item features onto a dense neural grid, generating robust similarity representations (J-Scores).
  - **K-Means++** takes these J-Scores and optimally groups the entire movie catalog into K distinct clusters (e.g., "Classic Drama", "90s Acton/Sci-Fi").
- **Strengths**: Provides a fast, scalable way to guarantee that final recommendations are contextually relevant to a user's known preferences.

How Overall Rating Prediction Happens

The final recommendation is generated through a carefully choreographed blending of the Collaborative and Content-based pathways, adjusting dynamically based on how much data is available for each specific user.

Step 1: Base Scoring

For a given user and a candidate item:

1. **CF Score**: The SVD model predicts the explicit numerical rating (e.g., 4.2 stars) based on matrix factorization.
2. **Content Score**: The KNN model looks at the user’s past positive ratings and calculates how semantically similar the candidate item is to those past favorites (using Cosine Similarity on the 140-dim feature vectors).

Both raw scores are then **normalized** (scaled between 0 and 1) so they can be blended fairly.

Step 2: Adaptive Alpha Blending

The core innovation is that the system doesn't trust Collaborative Filtering (CF) and Content equally for every user.

- If a user has 100 ratings, Collaborative Filtering is highly accurate and should be trusted.
- If a user has 2 ratings, Collaborative Filtering will be nearly random, so Content-Based matching should take over.

The system calculates a dynamic weight for the Collaborative Filtering score based on the user's rating count and a predefined cold start threshold (e.g., 20 ratings).

- **CF Weight** = (Number of User Ratings / Cold Start Threshold) * Alpha *(This ratio is capped at 1.0, and Alpha is a global hyperparameter typically set to 0.5)*
- **Content Weight** = 1.0 - CF Weight

The Role of Alpha (α) as a Tunable Hyperparameter

While the equation dynamically shifts trust between CF and Content at the *user level*, the global **Alpha** hyperparameter dictates the *overall ceiling* of that trust.

- If Alpha = 1.0, a power user relies 100% on Collaborative Filtering.
- If Alpha = 0.5, a power user receives exactly a 50/50 blend of CF and Content.

During Training (Hyperparameter Tuning): Because standard dataset compositions vary wildly, Alpha is rarely a "one-size-fits-all" number. It can be tuned dynamically during the training loop.

1. The model sweeps through a grid of Alpha values (e.g., 0.1, 0.3, 0.5, 0.7, 0.9).
2. For each Alpha, the model calculates the overall Hybrid Predictions for users in the **Validation Set** (the 15% of data held out specifically for tuning).
3. It evaluates the **Validation RMSE** (or Validation F1) for each Alpha.
4. The system permanently selects the Alpha that resulted in the lowest Validation Error, ensuring the best possible balance of CF vs Content for that specific dataset before the model ever touches the final Test Set.

The **Hybrid Score** is then calculated by blending them:

- **Hybrid Score** = (CF Weight * Normalized CF Score) + (Content Weight * Normalized Content Score)

Step 3: Cluster Filtering (The Guardrail)

Before generating the final list, the model looks at the user's historical ratings to find their **Preferred Cluster** (the specific K-Means cluster that contains the highest raw number of movies the user has rated positively).

1. The system takes the Top 200 items according to the blended **Hybrid Score**.
2. It applies a strict filter: **Only items belonging to the user's Preferred Cluster are kept.** This guarantees that the items are structurally and thematically aligned with the user's core tastes.

Step 4: Final Score and Ranking

Finally, the filtered list of candidates receives their **Final Score**, which is simply their original **Collaborative Filtering Score**. The list is sorted descendingly by this Final Score, and the Top-N items are returned to the user. This ensures that while content and structure guide the *candidate selection*, the final ranking is based on actual predicted user preference.

Fallback Mechanism: *If a user has 0 ratings, the system immediately bypasses this entire pipeline and returns the Top-N most globally popular items to begin gathering data.*

Evaluation Results

The hybrid model was evaluated across three Movielens datasets of varying sparsity and size.

Metric	MovieLens Small (ML-latest-small)	MovieLens 25M (ML-25m)	MovieLens 1M (ML-1m)
RMSE	0.9264	1.1361	0.9674

MAE	0.7142	0.8714	0.7668
Precision	0.0635	0.0016	0.0925
Recall	0.0733	0.0302	0.0926
F1	0.0681	0.0031	0.0925